Tarea 3

IIC3533 · Computación de Alto Rendimiento · 2026-2

Lunes 2 de Noviembre de 2026

- Trabajo en grupos de tres personas.
- Las pruebas iniciales se realizan en un computador personal y los experimentos principales en el *cluster* del curso.
- **Plazo de entrega:** Lunes 16 de noviembre de 2026, 23:59.

Factorización LU distribuida con MPI4PY

La factorización $A = LU$ de una matriz densa $A \in \mathbb{R}^{N \times N}$, con L triangular inferior con unos en la diagonal y U triangular superior, es el método directo estándar para resolver sistemas lineales, y cuesta $O(N^3)$ operaciones. En esta tarea se implementa sobre un *grid* 2D de procesos MPI, usando NumPy/SciPy para el cómputo local.

LU por bloques

Se divide A en bloques de $b \times b$ y se recorren los bloques diagonales $k = 0, 1, \ldots, N/b - 1$. En cada paso:

1. Factorizar el bloque diagonal: $A_{kk} = L_{kk} U_{kk}$.
2. Calcular los paneles: $U_{kj} = L_{kk}^{-1} A_{kj}$ para $j > k$, y $L_{ik} = A_{ik} U_{kk}^{-1}$ para $i > k$.
3. Actualizar: $A_{ij} \leftarrow A_{ij} - L_{ik} U_{kj}$ para $i, j > k$.

Los factores se guardan sobre la misma matriz. La matriz sobre la cual trabajaremos es diagonal dominante. Por eso estas iteraciones se completan sin intercambio de filas, que llevaría a la factorización $PA = LU$. En cada paso se usan las siguientes funciones:

- `scipy.linalg.lu_factor`: factorización del bloque diagonal.
- `scipy.linalg.solve_triangular`: cálculo de los paneles, resolviendo sistemas triangulares.
- Producto de matrices de NumPy (`@`): actualización de la submatriz.

Distribución bloque-cíclica

Se organiza un *grid* de $p_r \times p_c$ procesos y se crean comunicadores por filas y por columnas del *grid*:

```
pr, pc = MPI.Compute_dims(size, 2)
myrow, mycol = rank // pc, rank % pc
row_comm = comm.Split(myrow, mycol)
col_comm = comm.Split(mycol, myrow)
```

Los bloques se reparten como cartas: el bloque A_{IJ} pertenece al proceso (I mód p_r, J mód p_c). La Figura 1 muestra el caso de 6×6 bloques en un *grid* de procesos 2×3.

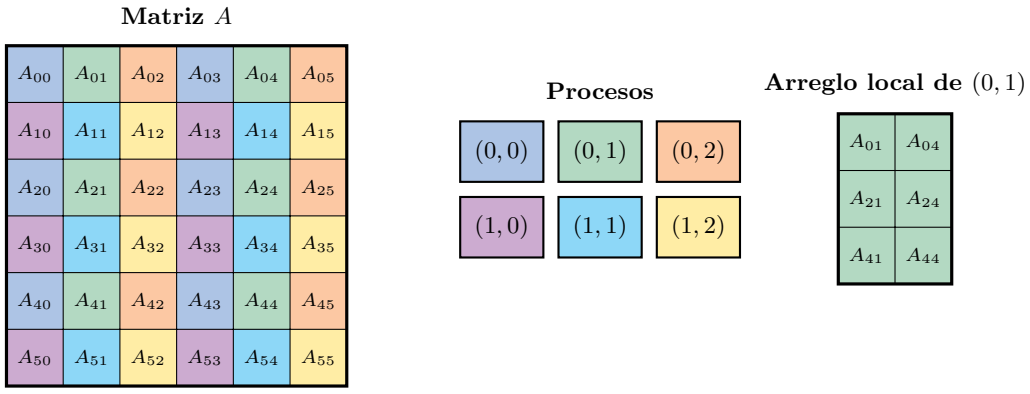


Figura 1: Distribución bloque-cíclica de 6×6 bloques en un *grid* de 2×3 procesos. Cada color corresponde a un proceso. A la derecha, los bloques que guarda el proceso $(0, 1)$.

Paso k distribuido. La Figura 2 muestra el paso $k = 2$ con la distribución anterior. Las columnas y filas de bloques anteriores a k ya tienen su valor final de L y U, y no participan.

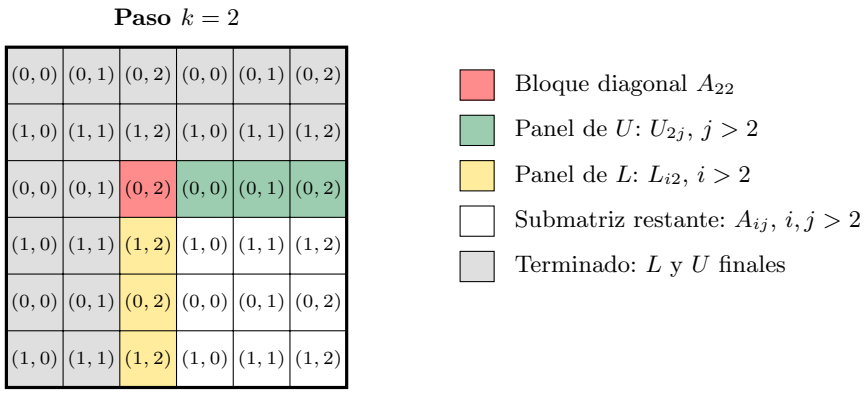


Figura 2: Paso $k = 2$ de la factorización con 6×6 bloques en un *grid* de procesos 2×3. Cada bloque indica el proceso que lo tiene.

1. **Bloque diagonal.** El bloque A_{22} pertenece al proceso $(0, 2)$. Este proceso lo factoriza y lo envía a los procesos de su fila del *grid*, $(0, 0)$ y $(0, 1)$, y de su columna, $(1, 2)$.
2. **Panel de U.** Los bloques a la derecha de A_{22}, U_{23}, U_{24} y U_{25}, pertenecen a los procesos $(0, 0)$, $(0, 1)$ y $(0, 2)$, respectivamente. Cada uno de estos procesos calcula su bloque.
3. **Panel de L.** De los bloques bajo A_{22}, L_{32} y L_{52} pertenecen al proceso $(1, 2)$ y L_{42} al proceso $(0, 2)$. Cada uno de estos procesos calcula sus bloques.
4. **Envío de paneles.** Los procesos $(0, 2)$ y $(1, 2)$ envían sus bloques L_{i2} a los procesos de su fila del *grid*. Los procesos $(0, 0)$, $(0, 1)$ y $(0, 2)$ envían sus bloques U_{2j} a los procesos de su columna.
5. **Actualización.** Cada proceso actualiza sus bloques de la submatriz restante. Por ejemplo, el proceso $(1, 1)$ tiene A_{34} y A_{54}. Recibió L_{32} y L_{52} desde $(1, 2)$ y U_{24} desde $(0, 1)$, y calcula $A_{ij} \leftarrow A_{ij} - L_{i2}U_{2j}$ para sus dos bloques.

Los bloques de cada proceso están ordenados por índice global. Por eso sus bloques de la submatriz restante forman un sub-arreglo contiguo del arreglo local. La actualización es entonces una sola multiplicación de matrices.

Caso general. En el paso k, sea $(r, c) = (k \bmod p_r,\ k \bmod p_c)$ el proceso que tiene A_{kk}:

- El proceso (r, c) factoriza A_{kk} y lo difunde en su fila de procesos (`row_comm`) y en su columna de procesos (`col_comm`).
- Los procesos de la fila r calculan sus bloques U_{kj} con $j > k$, y los de la columna c calculan sus bloques L_{ik} con $i > k$.
- En cada fila de procesos, el proceso de la columna c difunde sus bloques L_{ik} (`row_comm`). En cada columna de procesos, el proceso de la fila r difunde sus bloques U_{kj} (`col_comm`).
- Todos los procesos actualizan sus bloques A_{ij} con $i, j > k$.

Cluster

- Documentación: https://cluster.cenia.cl/. Ahí se encuentran ejemplos de cómo enviar *jobs* con `sbatch` y SLURM.
- Se recomienda usar Miniconda en el *cluster*.
- Acceso: agregar a `~/.ssh/config` la siguiente configuración, donde `<n>` es el número de su grupo, y conectarse con `ssh kraken`.

```
Host kraken
    HostName kraken.ing.puc.cl
    Port 33052
    User grupo<n>-iic3533
```

- Los *jobs* se envían con `sbatch`. Todo *job* debe incluir los siguientes campos. De lo contrario, no se ejecuta.

```
#SBATCH --qos=hpc-iic3533
#SBATCH --partition=hpc-iic3533
#SBATCH --account=default-account
```

- Para cada configuración es relevante definir:
  - `--ntasks`: número de procesos MPI, $p = p_r \times p_c$.
  - `--cpus-per-task`: *threads* por proceso, T.
  - `--nodes`: número de nodos.
  - `--ntasks-per-node`: procesos por nodo.
- Variables de ambiente relevantes dentro del *job*:
  - `SLURM_NTASKS`: número de procesos. Se usa en `mpirun -n`.
  - `SLURM_CPUS_PER_TASK`: *threads* por proceso.
  - `OMP_NUM_THREADS`, `OPENBLAS_NUM_THREADS` y `MKL_NUM_THREADS`: *threads* de BLAS que usan NumPy y SciPy. Deben fijarse en `$SLURM_CPUS_PER_TASK`.
- Nodos de la partición del curso: `ventress` y `yodaxico`.
- Límites por grupo:
  - Recursos: máximo 8 CPUs por nodo y 4 GB de RAM.
  - Duración máxima de un *job*: 15 minutos.
  - Máximo 4 *jobs* en cola y 1 *job* en ejecución.

Los experimentos en el *cluster* usan las siguientes configuraciones, con T *threads* por proceso:

Configuración	$p_r \times p_c$	T	**Nodos**	**Procesos por nodo**	**CPUs totales**
0	1×1	1, 2, 4, 8	1	1	1, 2, 4, 8
1	1×2, 2×1	1, 2, 4	1	2	2, 4, 8
2	2×2	1, 2	1	4	4, 8
3	2×4, 4×2	1	1	8	8
4	2×4	1	2	4	8
5	4×4	1	2	8	16

Preguntas

(a) **[10 pts]** En su computador, ejecute `lu_serial.py` con $N = 2^{12}$ y un *thread*, variando

$$b \in \{2^{12}, 2^{11}, 2^{10}, 2^9, 2^8, 2^7, 2^6\}.$$

Presente sus resultados en una tabla o gráfico y determine un valor apropiado de b según sus cálculos.

(b) **[20 pts]** Implemente `lu_distributed.py`, que reciba N, b, p_r y p_c, y mida por separado el tiempo de cómputo y de comunicación de cada proceso. Explique qué operación colectiva usa en cada paso, en qué comunicador y con qué raíz. Verifique la correctitud de su implementación respecto a la versión serial y explique cómo procedió.
Sugerencia: Puede ser útil (y más rápido) testear primeras versiones en su computador antes de pasar al cluster.
Observación: tal como en tareas previas, tenga precaución con el oversubscription.

(c) **[10 pts]** En el *cluster*, con $N = 2^{13}$, repita la variación de b de la parte (a) para la configuración 2 con $T = 2$. Presente sus resultados en una tabla o gráfico y comente si el valor de b elegido en (a) sigue siendo el mejor.

(d) **[20 pts]** En el *cluster*, con $N = 2^{13}$ y los valores de b obtenidos en (a) y en (c), ejecute las configuraciones 0 a 3. Calcule el *speedup* y la eficiencia. Presente sus resultados en una tabla o gráfico y justifique su elección de $T(1)$. ¿Qué configuración es la mejor? ¿Afecta la forma del *grid*?

(e) **[15 pts]** Con $N = 2^{13}$ y los valores de b obtenidos en (a) y en (c), ejecute las configuraciones 4 y 5. Compare la configuración 4 con el mismo *grid* en un nodo (configuración 3), separando cómputo y comunicación. Presente sus resultados en una tabla o gráfico y relacione la diferencia con la latencia y el ancho de banda dentro de un nodo y entre nodos.

(f) **[15 pts]** En el *grid* 2×2, compare $b = N/2$ y $b = N/16$, separando el tiempo de cómputo y de comunicación de cada proceso. Presente sus resultados en una tabla o gráfico. Explique los resultados obtenidos.

(g) **[10 pts]** Con el modelo de costo visto en clases (α, β, γ), estime el costo total de la factorización en función de N, b, p_r y p_c. ¿Qué términos explican el b óptimo que encontró? ¿Cuál es el cuello de botella según sus mediciones?

Instrucciones de entrega:

- Entregar por Canvas un `.zip` con los scripts SLURM y un documento PDF con sus resultados.
- El uso de IA está permitido, mientras esté debidamente declarado en el informe.
- Se evaluará la claridad y legibilidad de los gráficos y tablas. Es importante identificar títulos, *labels*, leyendas y tamaño de fuente adecuados.